# Data Preprocessing

1. Importing Libaries and Reading Cleaned Dataset
2. Label Encoding
    - Label Encoder
    - One hot Encoding
    - Dummies
3. Feature Scaling
    - MinMax Scaler
    - Standard Scaler
    - Binarizer
4. Feature Selection
    - SelectKBest
5. Feature Extraction
    - PCA
6. Train Test Split

In [22]:
```
import pandas as pd
import numpy as np

from sklearn.preprocessing import MinMaxScaler, StandardScaler, Binarizer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.feature_selection import SelectKBest, f_classif , chi2
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
```

In [55]:
```
df=pd.read_csv("salaries_cleaned.csv")
df.head()
```

Out[55]:

| | rank | discipline | yrs.since.phd | yrs.service | sex | salary |
|---|---|---|---|---|---|---|
| **0** | Prof | B | 19.0 | 18.0 | Male | 139750.0 |
| **1** | Prof | B | 20.0 | 16.0 | Male | 173200.0 |
| **2** | AsstProf | B | 4.0 | 3.0 | Male | 79750.0 |
| **3** | Prof | B | 45.0 | 39.0 | Male | 115000.0 |
| **4** | Prof | B | 40.0 | 41.0 | Male | 141500.0 |

In [24]:
```
df.shape
```

Out[24]:
```
(390, 6)
```

## Label Encoding

In [56]:
```
le=LabelEncoder()
```

In [57]:
```
categorical_cols=df.select_dtypes(exclude=np.number).columns
```

In [58]:
```
for col in categorical_cols:
    df[col]=le.fit_transform(df[col])
```

In [30]:
```
df.head()
```

Out[30]:

| | rank | discipline | yrs.since.phd | yrs.service | sex | salary |
|---|---|---|---|---|---|---|
| **0** | 2 | 1 | 19.0 | 18.0 | 1 | 139750.0 |
| **1** | 2 | 1 | 20.0 | 16.0 | 1 | 173200.0 |
| **2** | 1 | 1 | 4.0 | 3.0 | 1 | 79750.0 |
| **3** | 2 | 1 | 45.0 | 39.0 | 1 | 115000.0 |
| **4** | 2 | 1 | 40.0 | 41.0 | 1 | 141500.0 |

## One hot Encoding

In [33]:
```
ohe = OneHotEncoder(sparse_output=False, drop=None)
encoded_array = ohe.fit_transform(df[["rank", "discipline", "sex"]])
encoded_columns = ohe.get_feature_names_out(["rank", "discipline", "sex"])
df_ohe = pd.DataFrame(encoded_array, columns=encoded_columns)
```

In [34]:
```
df_ohe
```

Out[34]:

| | rank_0 | rank_1 | rank_2 | discipline_0 | discipline_1 | sex_0 | sex_1 |
|---|---|---|---|---|---|---|---|
| **0** | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 |
| **1** | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 |
| **2** | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 |
| **3** | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 |
| **4** | 0.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **385** | 0.0 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 1.0 |
| **386** | 0.0 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 1.0 |
| **387** | 0.0 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 1.0 |
| **388** | 0.0 | 0.0 | 1.0 | 1.0 | 0.0 | 0.0 | 1.0 |
| **389** | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 |

390 rows × 7 columns

### Dummies

In [41]:
```
df_dummies = pd.get_dummies(df, columns=["rank", "discipline", "sex"])
df_dummies.head()
```

Out[41]:

| | yrs.since.phd | yrs.service | salary | rank_AssocProf | rank_AsstProf | rank_Prof |
|---|---|---|---|---|---|---|
| **0** | 19.0 | 18.0 | 139750.0 | False | False | True |
| **1** | 20.0 | 16.0 | 173200.0 | False | False | True |
| **2** | 4.0 | 3.0 | 79750.0 | False | True | False |
| **3** | 45.0 | 39.0 | 115000.0 | False | False | True |
| **4** | 40.0 | 41.0 | 141500.0 | False | False | True |

## Feature Scaling

### Minmax Scaler

In [59]:
```
numerical_cols=df.select_dtypes(include=np.number).columns
# numerical_cols=["yrs.since.phd", "yrs.service", "salary"]
minmax = MinMaxScaler()
for col in numerical_cols:
    df[col]=minmax.fit_transform(df[[col]])
```

In [45]:
```
df.head()
```

Out[45]:

| | rank | discipline | yrs.since.phd | yrs.service | sex | salary |
|---|---|---|---|---|---|---|
| **0** | Prof | B | 0.327273 | 0.300000 | Male | 0.598175 |
| **1** | Prof | B | 0.345455 | 0.266667 | Male | 0.842336 |
| **2** | AsstProf | B | 0.054545 | 0.050000 | Male | 0.160219 |
| **3** | Prof | B | 0.800000 | 0.650000 | Male | 0.417518 |
| **4** | Prof | B | 0.709091 | 0.683333 | Male | 0.610949 |

Standard Scaler

In [46]:
```
stdscaler= StandardScaler()
for col in numerical_cols:
    df[col]=stdscaler.fit_transform(df[[col]])
```

In [47]:
```
df.head()
```

Out[47]:

| | rank | discipline | yrs.since.phd | yrs.service | sex | salary |
|---|---|---|---|---|---|---|
| **0** | Prof | B | -0.265917 | 0.022882 | Male | 0.912911 |
| **1** | Prof | B | -0.187647 | -0.132316 | Male | 2.061497 |
| **2** | AsstProf | B | -1.439964 | -1.141101 | Male | -1.147334 |
| **3** | Prof | B | 1.769098 | 1.652457 | Male | 0.063060 |
| **4** | Prof | B | 1.377749 | 1.807655 | Male | 0.973001 |

Binarizer

In [48]:
```
binarizer = Binarizer(threshold=0.1)
df["salary_binarized"] =binarizer.fit_transform(df[["salary"]])
df.head()
```

Out[48]:

| | rank | discipline | yrs.since.phd | yrs.service | sex | salary | salary_binarize |
|---|---|---|---|---|---|---|---|
| **0** | Prof | B | -0.265917 | 0.022882 | Male | 0.912911 | 1. |
| **1** | Prof | B | -0.187647 | -0.132316 | Male | 2.061497 | 1. |
| **2** | AsstProf | B | -1.439964 | -1.141101 | Male | -1.147334 | 0. |
| **3** | Prof | B | 1.769098 | 1.652457 | Male | 0.063060 | 0. |
| **4** | Prof | B | 1.377749 | 1.807655 | Male | 0.973001 | 1. |

## Feature Selection

### Select K Best (for Classification)

In [71]:
```
selector = SelectKBest(score_func=f_classif, k=3) # can also use chi2
x=df[["discipline","yrs.since.phd","yrs.service","sex","salary"]] # input feat
y=df["rank"] # target feature
kfeat = selector.fit_transform(x,y)
selected_features = x.columns[selector.get_support()]
print(selected_features)
print(selector.scores_)
```

```
Index(['yrs.since.phd', 'yrs.service', 'salary'], dtype='object')
[  1.47068813 171.39729386 109.80381812   4.75232646 129.00594863]
```

### Select K Best (for Regression)

In [72]:
```
from sklearn.feature_selection import SelectKBest, f_regression
selector = SelectKBest(score_func=f_regression, k=3)
x=df[["rank","discipline","yrs.since.phd","yrs.service","sex"]]
y=df["salary"]
kfeat = selector.fit_transform(x,y)
selected_features = x.columns[selector.get_support()]
print(selected_features)
print(selector.scores_)
```

```
Index(['rank', 'yrs.since.phd', 'yrs.service'], dtype='object')
[146.17685734  12.98540764  76.28507503  43.97740159   7.66599834]
```

## Feature Extraction

### PCA

In [74]:
```
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x)
```

In [77]:
```
print(pca.explained_variance_ratio_) # higher the better
```

```
[0.44633211 0.30411185]
```

## Train Test Split

In [79]:
```
xtrain,xtest,ytrain,ytest=train_test_split(x_pca,y,test_size=0.2,shuffle=True,
print(xtrain.shape)
print(xtest.shape)
print(ytrain.shape)
print(ytest.shape)
```

```
(312, 2)
(78, 2)
(312,)
(78,)
```

In [ ]: